Proofline

Your rules.
Checked on every change.

A portable test harness: a reusable system for running checks and keeping evidence as your application changes.

Author once.
Repeat with confidence.

Turn an important rule into a check the whole team can repeat.

01

Define the rule

Name the behavior and the result that must stay true.

→ 02

Write the test

Write and review deterministic tests with clear ownership.

→ 03

Repeat on changes

Run checks on changes. Retain results and diagnostics.

Continuous integration (CI) reruns these tests automatically. No AI model or agent runs during test execution.

Small checks. Real journeys.

536

Focused tests · Vitest

Rules, calculations, APIs, permissions, retries and interface behavior.

43

Browser tests · Chromium

Real navigation across screens, plus scenario and containment checks.

Your app → Contracts + runners → Focused + browser checks → Reports + evidence

36 critical rules map to exact tests. Snapshot: 49 unit files across 11 targets (one empty).
The 43 browser tests include 24 scenario tests.

A deposit must not earn commission

A $25 deposit should earn $0 commission. A controlled source change breaks that rule.
Replay real captured runs from an isolated copy. A developer applies the fix.

Bug present

$2.50

Wrong commission on the $25 deposit.
Existing test: FAIL · exit 1

```
AssertionError: expected [ { …(8) } ] to deeply equal []
+     "amountCents": 250,
```

After the developer fix

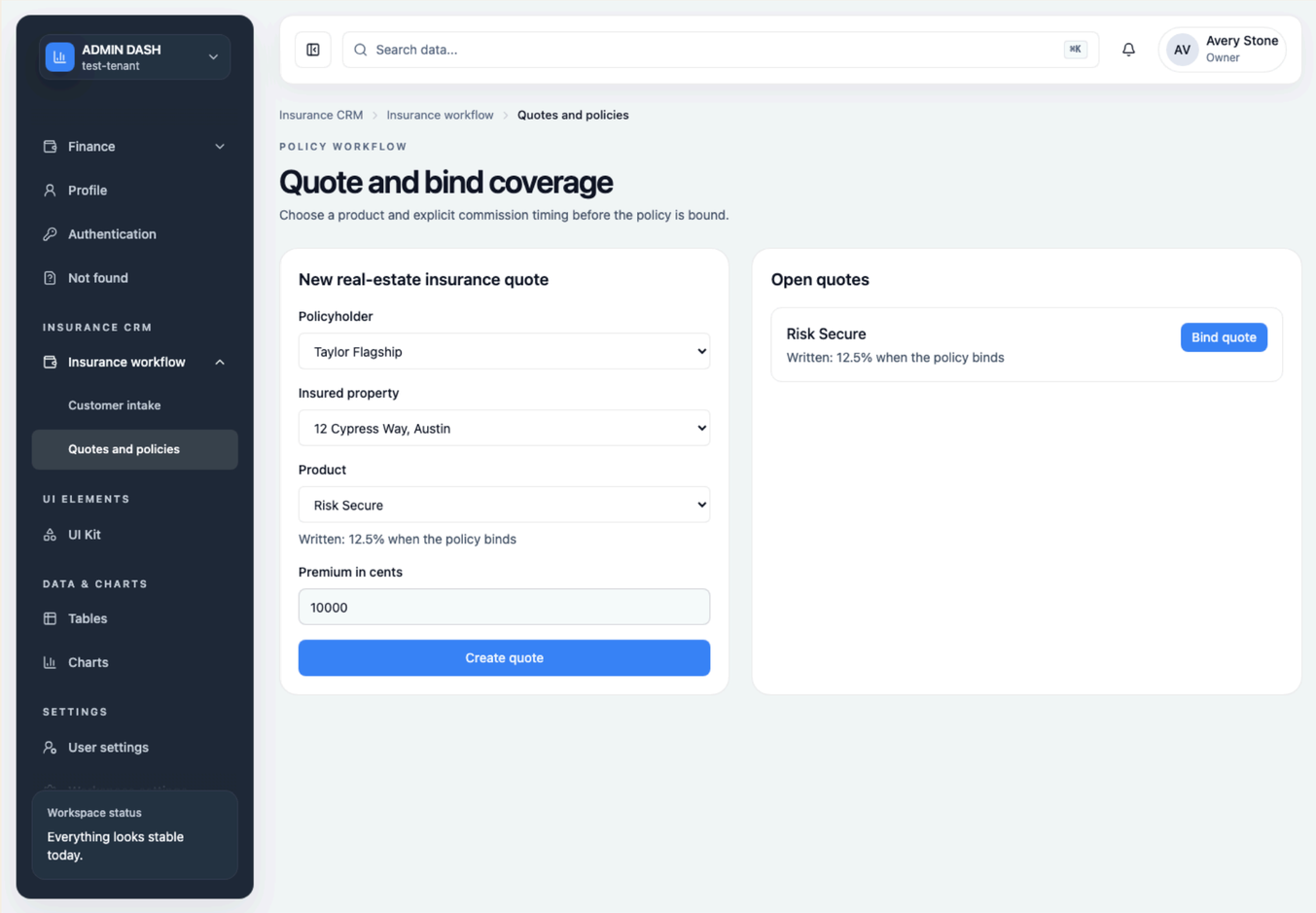
$0.00

Correct commission on the deposit.
Same unchanged test: PASS · exit 0

```
 Test Files  1 passed (1)
      Tests  1 passed | 23 skipped (24)
```

Captured execution, not a live test in this slide. The test was unchanged. The fixed finance suite passed 24 / 24.
Actual run results · Reproduce this controlled regression · Source change

Follow a real user journey.



01 **Sign in**

A protected link requires login.

02 **Create the customer**

Add their insured property.

03 **Quote and bind**

Use visible controls to create coverage.

04 **Open finance**

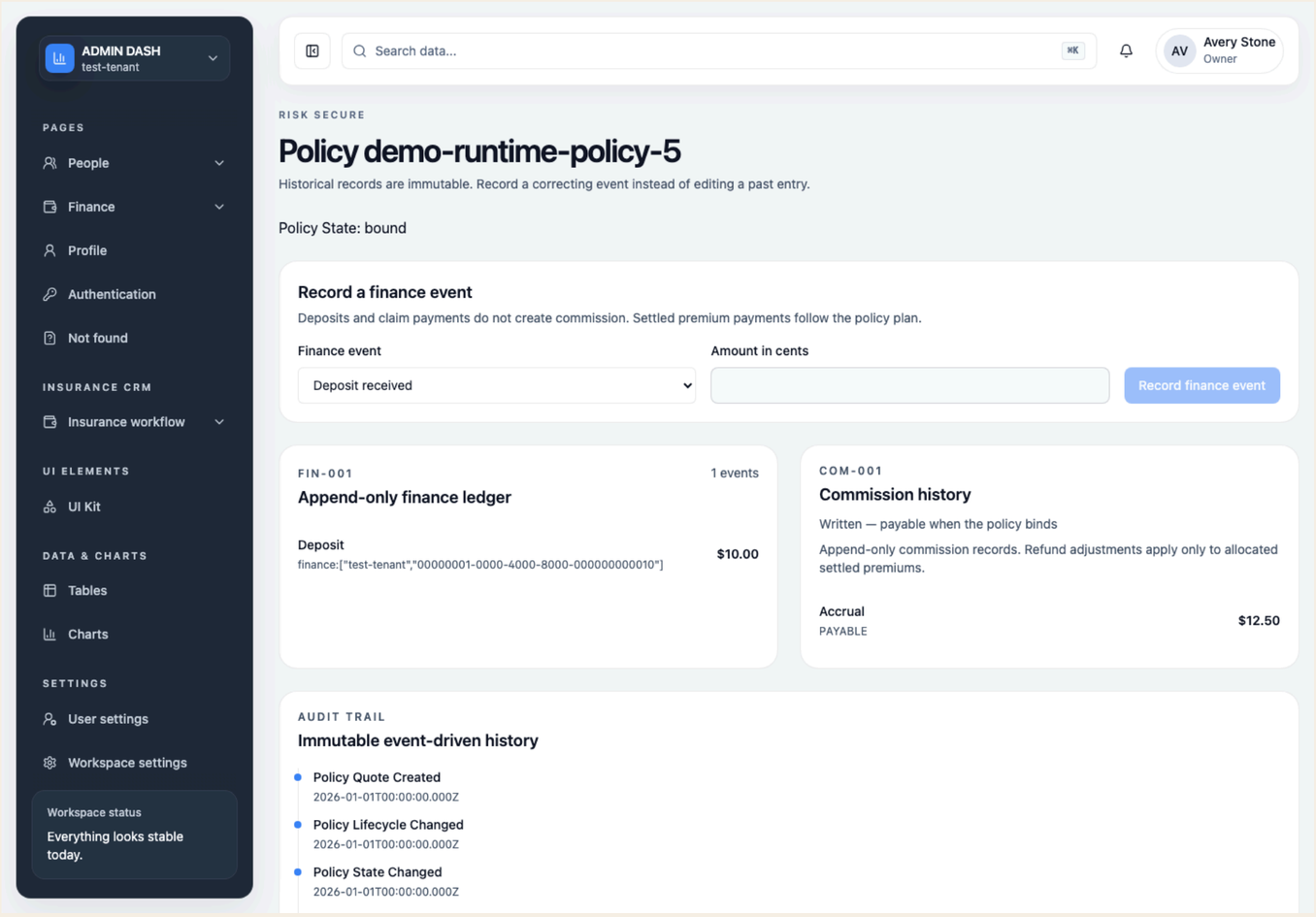
Record a deposit; inspect the ledger.

05 **Log out and revisit**

The protected policy is denied.

Static frame. Local test build of the reference CRM · synthetic data · Play the recording in the browser edition

The screen is half the proof.



1 bound policy

1 finance ledger

0 pending messages

After the visible actions, the test inspects persisted state.

Logout also revokes the authenticated session.

The recorded run also retained one customer, property, quote, finance entry and commission entry, with zero unexpected requests.

Trust includes the difficult paths.

01

Right access

Roles and tenants control which routes and records a user can reach.

02

Safe recovery

Retrying after a lost response does not duplicate the policy or ledger.

03

Isolated runs

Fresh state and deterministic fakes. Test adapters stay out of production.

Missing or ambiguous evidence fails closed.

Four stronger guarantees.

Each check leaves evidence you can inspect and trust.

01 Required tests really pass

Skipped, missing or expected-failure results stop the gate.

02 Failures stay inspectable

Logs, traces, screenshots and videos remain with each run.

03 Release the verified build

Source, run identity and file hashes bind the exact output.

04 Proven on a second app

The work-orders app passes with 31 core files unchanged.

Deterministic checks. Independent review. Missing evidence stops progress.

Release the build that passed.

The release workflow checks the artifact again before deployment.

01 → 02 → 03 → 04

Exact source

One committed revision, repository and run.

Complete checks

Required suites, coverage and second-app validation.

Audited build

Production output excludes test adapters.

Verified artifact

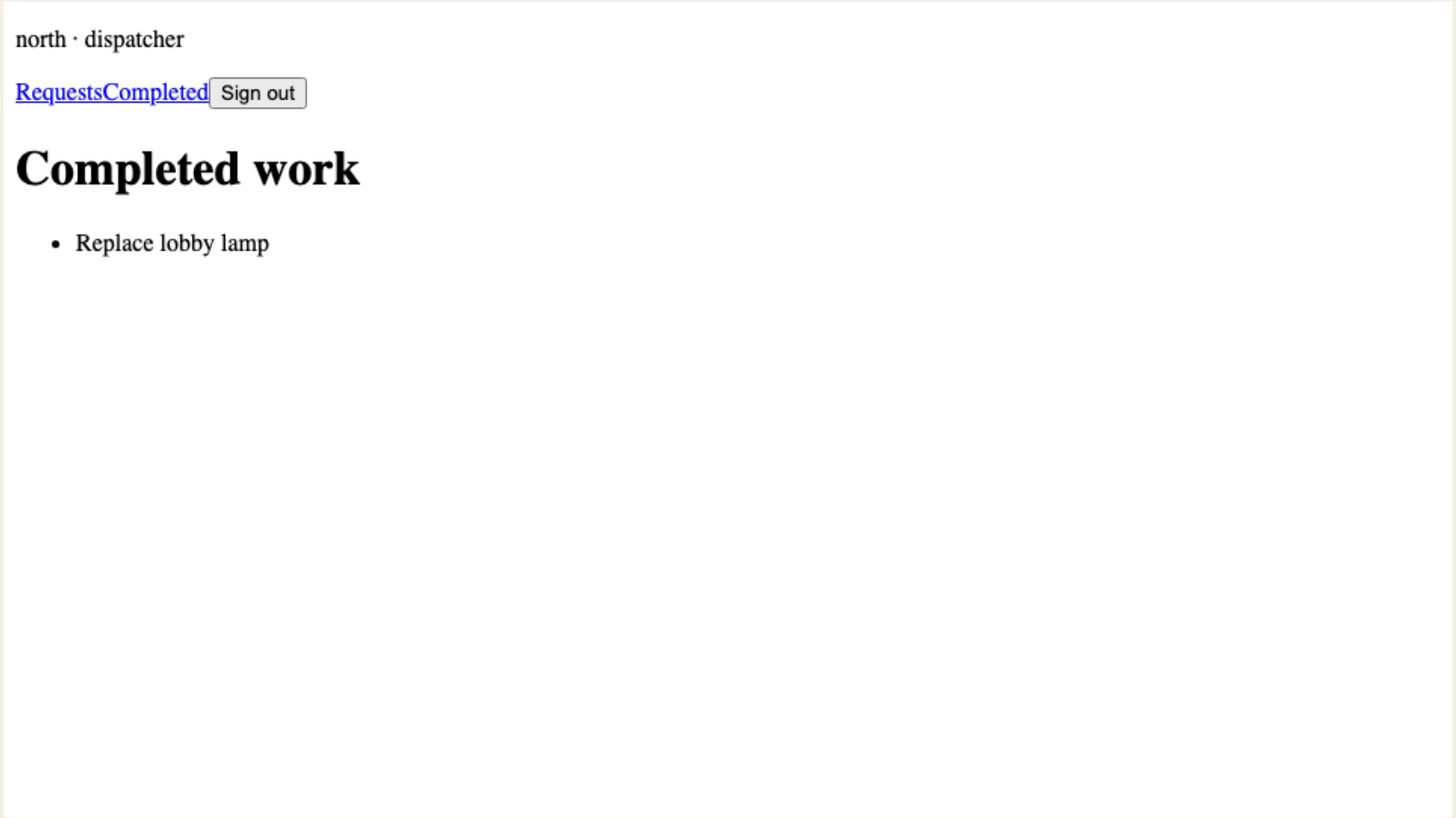
Every file hash checked. Deploy without rebuilding.

664 files verified in the local release bundle
Directory and ZIP checks agree.

Actual local packaging and verification passed. Hosted GitHub and Vercel deployment were not run.

The same harness. Another app.

A separate work-orders app owns its routes, rules, login and fake APIs.



Static frame. Recorded guarded test build · deterministic fake APIs · Capture and adaptation evidence

01 **Sign in to Requests**

A dispatcher sees their organization's work.

02 **Complete the request**

Open Completed through visible navigation.

03 **Sign out and revisit**

The protected screen requires login again.

31 core files copied without changes

Coverage you can inspect.

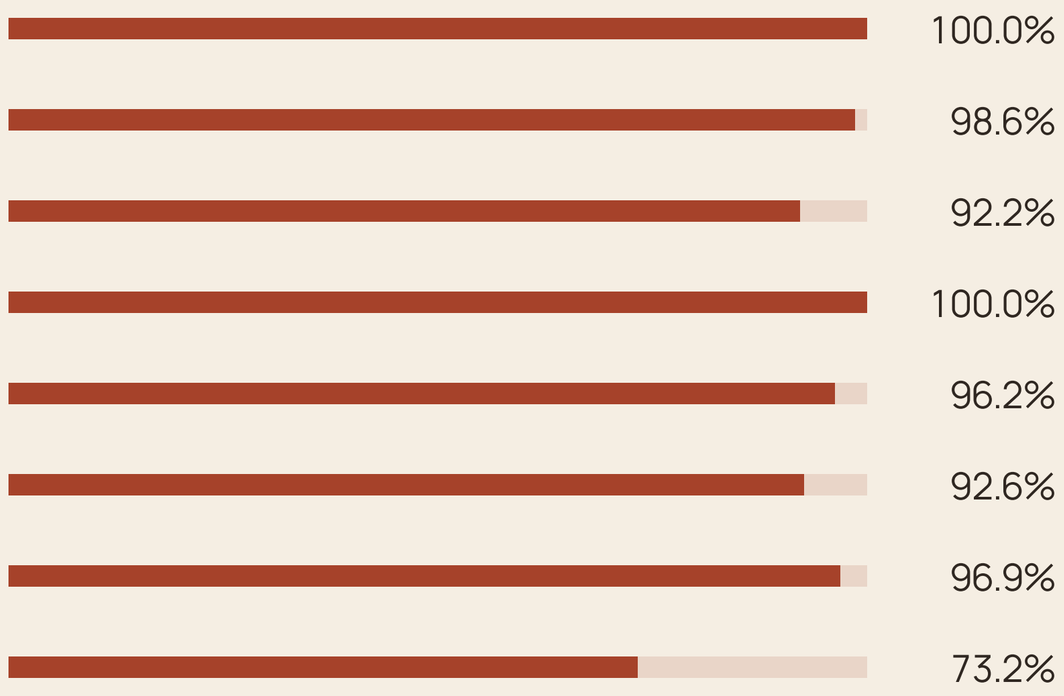
8 / 8

Governed groups meet their gates

Approved minimums can hold or rise.

A passing gate does not mean 100% coverage.

Exercised lines / classified source



23 classified files; covered ÷ total lines. This is a subset, not global coverage. The gate checks each file across four metrics.

Harness + release tools: parent coverage captures only part of subprocess execution; guarded behavior is tested separately.

The latest verification results.

A full local run from cleared build, cache and report outputs.

18 / 18

Verification steps passed

Includes test execution, governance, portability and release bundle checks.

536	**Unit tests passed** 49 test files across 11 targets
43	**Chromium tests passed** Separate scenario: 24 · shards: 12 + 12
8 / 8	**Coverage groups passed** 23 classified files · prior minimums preserved
10 / 10	**Second-app gates passed** Includes 4 browser workflows and production checks

Scenario and shard runs repeat tests from the full browser suite. Counts overlap.
Local CI-contract validation; no hosted workflow or deployment claimed. Measured results and sources

Bring the proof to your app.

Reuse Proofline

Selection · verified test outcomes
Coverage gates · retained evidence



Connect your application

Routes · business rules · auth
APIs · deterministic provider fakes

Proven on the reference CRM and an independent work-orders app.
Connect your routes, rules and adapters. Keep the core unchanged.